



Procesadores del Lenguaje
Grado en Ingeniería Informática

Desarrollo de traductores de c a lisp y de lisp a forth

Miembros del grupo:

Alejandro Quirante Sanz - 100522183@alumnos.uc3m.es
Carlos Martín Gallardo - 100522258@alumnos.uc3m.es

Grupo 412

2025/2026

Índice

1. Declaración de uso de herramientas	2
2. Introducción	2
3. Diseño del Frontend (trad.y)	2
3.1. Estructura jerárquica de la gramática	2
3.2. Declaraciones de variables globales	3
3.3. Variables locales y tabla de símbolos	3
3.4. Definición de funciones	4
3.4.1. Sentencia return	5
3.5. Asignación	5
3.6. Sentencias de control de flujo	6
3.6.1. Sentencias condicionales: if-else	6
3.6.2. Bucles	6
3.6.2.1. Bucles: while	6
3.6.2.2. Bucle for	6
3.6.2.3. Estructura switch-case	7
3.7. Impresión con printf y puts	7
3.8. Expresiones y operadores	8
3.9. Vectores (arrays)	9
3.10. Directiva <code>//@ (main)</code>	9
3.11. Gestión de memoria y código diferido	10
4. Diseño del Backend (back.y)	10
4.1. Traducción de variables y asignaciones	10
4.2. Funciones	11
4.3. Estructuras de control	12
4.4. Llamadas a funciones	12
4.4.1. If	13
4.4.2. Loop while	13
4.5. Operadores	14
4.6. Impresión	14
4.7. Consideraciones sobre la pila	15
5. Pruebas	15
5.1. Conjunto de pruebas	15
5.2. Resultados de las pruebas	16
6. Conclusiones	16
7. Listado de pruebas y código generado	17

1. Declaración de uso de herramientas

Durante la realización de esta práctica no se ha utilizado inteligencia artificial generativa para escribir código de los traductores ni para generar las gramáticas. El trabajo ha sido realizado íntegramente por los dos autores, con una participación equitativa: Alejandro Quirante Sanz se ha encargado del frontend en mayor medida y Carlos Martín Gallardo del backend, aunque ambos han contribuido en la realización total de la práctica, asistiendo a las clases obligatorias y realizando el trabajo parte por parte como se ha requerido semana a semana y colaborado en la depuración y en las pruebas. El trabajo sigue las directrices establecidas en el enunciado de la práctica final, incluyendo la obligatoriedad de no generar automáticamente la llamada a la función `main`, el uso de la directiva `//@ (main)` para activar la ejecución, y la diferenciación entre variables globales y locales mediante concatenación del nombre de la función.

2. Introducción

El objetivo de esta práctica es la construcción de un traductor completo que permita convertir un subconjunto significativo del lenguaje C en código ejecutable en el lenguaje Forth, utilizando Lisp como representación intermedia. El trabajo se ha estructurado en dos traductores claramente diferenciados:

- **Frontend** (implementado en el fichero `trad.y`): analiza un programa fuente escrito en C (siguiendo las restricciones detalladas en el enunciado) que traduce el código inicial de c a Lisp. Este Lisp resultante actúa como código intermedio.
- **Backend** (implementado en el fichero `back.y`): toma el código Lisp generado por el frontend y lo traduce a notación postfija, propia del lenguaje Forth.

Ambos traductores se han desarrollado utilizando la herramienta Bison, especificando una gramática que permite reconocer el subconjunto inicial de c junto con acciones semánticas que generan el código resultante.

A continuación se presenta el diseño del frontend, el backend, la integración de ambos y los resultados de las pruebas.

3. Diseño del Frontend (`trad.y`)

El frontend es el componente encargado de analizar el código C de entrada y traducirlo a Lisp. El fichero `trad.y` contiene la especificación de la gramática, las acciones semánticas asociadas y funciones auxiliares para la gestión de memoria y la tabla de símbolos locales.

3.1. Estructura jerárquica de la gramática

Se ha optado por una gramática jerárquica y recursiva por la derecha para evitar conflictos LALR y facilitar la lectura. El axioma `programa` deriva en dos grandes bloques: primero las declaraciones globales y después las definiciones de funciones. Esta separación es importante porque en Lisp las funciones deben definirse antes de ser utilizadas, y al ordenar las declaraciones globales (variables) al principio se simplifica la generación de código.

```
programa:      declaraciones_globales      { printf("%s", $1.code); }
              definiciones_funciones      { ; }
              ;
declaraciones_globales:      { sprintf(temp, ""); $$code
= gen_code(temp); }
              | declaracion_global
```

```

    declaraciones_globales      { sprintf(temp, "%s%s", $1.code, $2.code);
                                $.code = gen_code(temp); }
    ;

    declaracion_global: INTEGER lista_global ';'      { $.code = $2.code; }
    ;

```

3.2. Declaraciones de variables globales

Las variables globales se declaran al principio del código, permitiendo únicamente la declaración de variables `int` y permitiendo así mismo la declaración de variables encadenada.

```

int a, b, c;
// 0
int a = 0;

```

Según el subconjunto de C considerado, una variable global se declara con `int <identificador>;` y opcionalmente se puede inicializar con una constante numérica: `int <id> = <numero>;`. También se permiten declaraciones múltiples separadas por comas: `int a, b = 5, c;`. La gramática contempla estas posibilidades mediante los no terminales `lista_global` e `init_global`.

```

    declaracion_global: INTEGER lista_global ';'      { $.code = $2.code; }
    ;

    lista_global: init_global                          { $.code = $1.code; }
    | init_global ',' lista_global      { sprintf(temp, "%s%s", $1.code, $3.code);
                                        $.code = gen_code(temp); }
    ;

    init_global: IDENTIF                                { sprintf(temp, "(setq %s
0)\n", $1.code);
                                                        $.code = gen_code(temp); }
    | IDENTIF '=' NUMBER                                { sprintf(temp, "(setq %s %d)\n",
$1.code, $3.value);
                                                        $.code = gen_code(temp); }
    | IDENTIF '[' NUMBER ']'                            { sprintf(temp, "(setq %s (make-
array %d))\n", $1.code, $3.value);
                                                        $.code = gen_code(temp); }
    ;

```

La traducción de una variable global es `(setq nombre valor)`. En el caso de vectores, se emplea `(setq nombre (make-array tamaño))`. Se ha respetado la condición de que las inicializaciones solo pueden ser constantes numéricas, nunca expresiones, ya que en C las variables globales se inicializan en tiempo de compilación.

3.3. Variables locales y tabla de símbolos

Las variables declaradas dentro del cuerpo de una función son locales a esa función. En Lisp, las variables definidas con `setq` son globales por defecto, por lo que para simular el ámbito local hemos cambiado el nombre de la variable, concatenando el nombre de la función con el nombre

de la variable original. Por ejemplo, la variable `x` dentro de la función `main` se traduce a `main_x`. De esta forma se evita colisiones con variables globales del mismo nombre.

Para decidir si una variable utilizada en una expresión es local o global, se mantiene una tabla de símbolos locales (`local_var_table`) que se actualiza cada vez que se declara una variable local. Cuando se traduce una asignación o un acceso a variable, se consulta la tabla y se genera el nombre adecuado.

```
typedef struct s_local_var {
    char *name;
    struct s_local_var *next;
} t_local_var;
t_local_var *local_var_table = NULL;

void add_local_var(char *name) { ... }
int is_local_var(char *name) { ... }
char *concat_with_function(char *var_name) {
    if (is_local_var(var_name)) {
        sprintf(temp, "%s_%s", current_function, var_name);
        return gen_code(temp);
    } else return var_name;
}
```

La variable global `current_function` almacena el nombre de la función que se está procesando. Al entrar en una función, se limpia la tabla de locales (con `clear_local_vars()`). Los parámetros de la función también se consideran variables locales y se añaden a la tabla.

3.4. Definición de funciones

La gramática reconoce funciones sin especificar el tipo de retorno, por lo que en `c`, se deduce que el tipo a devolver es `int` de forma implícita. Para simplificar, no exigimos la palabra `int` delante del nombre de la función; el no terminal `definicion_funcion` comienza directamente con el identificador de la función o con `MAIN`.

`Main` requiere de tratamiento especial ya que no admite parámetros. Para el resto de funciones que si los requieren, los parámetros se declaran con `int id` y separados por comas. El cuerpo de la función es un bloque entre llaves.

```
definiciones_funciones: definicion_funcion
                        definiciones_funciones { ; }
                        |
                        ;

definicion_funcion: MAIN
                  { strcpy(current_function, "main");
                    printf("(defun main ())"); }
                  r_definicion_main
                  | IDENTIF
                  { strcpy(current_function, $1.code);
                    printf("(defun %s", $1.code); }
                  r_definicion_resto
                  ;

r_definicion_main: '(' ')' bloque
                  { printf("\n %s\n", $3.code);
                    clear_local_vars(); }
```

```

;
r_definicion_resto: '(' parametros ')' bloque { printf(" (%s) %s)\n", $2.code,
$4.code);
clear_local_vars(); }
;

```

3.4.1. Sentencia return

El return puede aparecer en cualquier punto de una función. Se traduce a (return-from nombre-funcion expr), que, aunque no sea la mejor forma de devolver valores en Lisp, es la forma de devolver valores de forma no estructurada como en c. Además en c se supone que el tipo devuelto será un int, sin embargo, estas comprobaciones no las podemos hacer ya que no implementamos análisis semántico.

```

return_stmt: RETURN expresion
{ sprintf(temp, "(return-from %s %s) ", current_function, $2.code); ... }
| RETURN { sprintf(temp, "(return-from %s) ", current_function); ... }

```

La traducción de la función comienza con (defun nombre (parámetros), luego se imprime el código del bloque (que ya incluye las declaraciones locales y sentencias) y se cierra con un paréntesis. Los parámetros se concatenan separados por espacios.

Ejemplo:

```

(defun factorial ( factorial_n )
  setf factorial_n 7)
  (setf resultado 1)
  (loop while (> factorial_n 1) do
    (setf resultado (* resultado factorial_n))
    (setf factorial_n (- factorial_n 1))
  )
  (return-from factorial (princ resultado))

```

3.5. Asignación

La asignación en C (por ejemplo, a = b + 1;) se traduce a (setf a (+ b 1)). Se ha elegido setf en lugar de setq para facilitar la distinción semánticamente la asignación de la declaración con inicialización (que usa setq). Además, se soporta la asignación a elementos de vectores: v[i] = expr se traduce a (setf (aref v i) expr).

```

asignacion: IDENTIF '=' expresion { char *var_name =
concat_with_function($1.code);
sprintf(temp, "(setf %s %s) \n",
var_name, $3.code);
$$code = gen_code(temp); }
| IDENTIF '[' expresion ']' '=' { char *var_name
expresion { char *var_name
= concat_with_function($1.code);
sprintf(temp, "(setf (aref %s %s)
%s) ", var_name, $3.code, $6.code);
$$code = gen_code(temp); }
;

```

3.6. Sentencias de control de flujo

3.6.1. Sentencias condicionales: if-else

La estructura condicional `if` en C puede presentar el problema del dangling `else` si las ramas no usan llaves. Para evitarlo, en esta práctica se exige que tanto la rama `then` como la rama `else` vayan siempre entre llaves. De esta forma, la gramática puede diferenciar claramente dónde termina cada bloque. La traducción a Lisp utiliza `(if cond (progn ...) (progn ...))`, empaquetando las sentencias del bloque con `progn` para permitir múltiples expresiones. Como medida adicional para asegurar que el último bloque `else` se empareje con el último `if`, incluimos la directiva de bison `%prec`.

```
if_stmt: IF '(' expresion ')' bloque %prec ELSE { sprintf(temp, "(if %s (progn %s))\n", $3.code, $5.code);
                                                $.code = gen_code(temp); }
        | IF '(' expresion ')' bloque
          ELSE bloque
          (progn %s)) \n", $3.code, $5.code, $7.code);
                                                { sprintf(temp, "(if %s (progn %s)\n", $3.code, $5.code, $7.code);
                                                $.code = gen_code(temp); }
        ;
```

3.6.2. Bucles

3.6.2.1. Bucles: while

El bucle `while` de c: `while (expr) { ... }` se traduce a `(loop while expr do ...)` en Lisp. La expresión condicional se evalúa al comienzo de cada iteración.

```
while_stmt: WHILE '(' expresion ')' bloque { sprintf(temp, "(loop while %s do\n%s)", $3.code, $5.code);
                                                $.code = gen_code(temp); }
        ;
```

3.6.2.2. Bucle for

El bucle `for` se admite únicamente en su forma más sencilla: `for (inicializacion; condicion; postcondición) { ... }`, donde la inicialización es una asignación simple, la condición es una expresión que se evalúa a verdadero/falso (distinto de cero) y la postcondición es, en este caso una llamada a una macro `INC(x)` o `DEC(x)`. La traducción expande el bucle en una estructura `loop while` de Lisp que ejecuta primero la inicialización, luego evalúa la condición, ejecuta el cuerpo y finalmente realiza el incremento.

```
for_stmt: FOR '(' for_inicial ';' for_condicion ';'
            for_incremento ')' bloque { sprintf(temp, "%s\n(loop while %s\ndo %s %s\n)", $3.code, $5.code, $9.code, $7.code);
                                        $.code = gen_code(temp); }
        ;

for_inicial: asignacion { $$ = $1; }
            |           { $.code = gen_code(""); }
            ;
```

```

for_condicion: expresion          { $$ = $1; }
              |
              ;                   { $$code = gen_code("1"); }

for_incremento: inc_dec          { $$ = $1; }
              |
              ;                   { $$code = gen_code(""); }

```

3.6.2.3. Estructura switch-case

La sentencia `switch` se traduce a la construcción `case` de Lisp. La sintaxis de entrada debe tener cada `case` finalizado con `break`, esto no es necesario en lisp, por lo que el `break` únicamente se elimina. La cláusula `default` se convierte en `otherwise`.

```

switch_stmt: SWITCH '(' expresion ')' switch_bloque { sprintf(temp, "(case %s\n%s)",
$3.code, $5.code);
                                                    $$code = gen_code(temp); }

switch_bloque: '{' lista_case default_case '}' { sprintf(temp, "%s%s", $2.code,
$3.code);
                                                    $$code = gen_code(temp); };

lista_case:
    | case_item lista_case
    { $$code = gen_code(""); }
    { sprintf(temp, "%s\n%s",
$1.code, $2.code);
      $$code = gen_code(temp); };

case_item: CASE expresion ':' lista_sentencias { sprintf(temp, " (%s %s)", $2.code,
$4.code);
                                                    $$code = gen_code(temp); };

default_case:
    | DEFAULT ':' lista_sentencias
    { $$code = gen_code(""); }
    { sprintf(temp, " (otherwise
%s)", $3.code);
      $$code = gen_code(temp); };

```

3.7. Impresión con printf y puts

En C existen `printf` y `puts` para imprimir por la salida estándar. En esta práctica se simplifica el tratamiento de `printf`:

- La cadena de formato se ignora (aunque se reconoce como token `STRING`).
- Cada argumento adicional (expresión o cadena) se traduce a `(princ ...)`.
- `puts(cadena)` se traduce a `(print cadena)`, que añade un salto de línea final.
- `printf` con solo una cadena (sin argumentos) se traduce a `(print cadena)`.

De esta forma, la salida producida por Lisp es similar a la del programa C original, aunque con posibles diferencias en los saltos de línea que son aceptables según el enunciado.

```

print_funcion: printf_funcion      { ; }

```

<pre> puts_funcion ; printf_funcion: PRINTF '(' STRING ')' \$3.code); PRINTF '(' STRING ',' argumentos_printf ')' ; argumentos_printf: argumento_printf argumento_printf ',' argumentos_printf \$1.code, \$3.code); ; argumento_printf: expresion STRING \"%s\\\"", \$1.code); ; puts_funcion: PUTS '(' STRING ')' \\n", \$3.code); PUTS '(' expresion ')' \\n", \$3.code); ; </pre>	<pre> { ; } { sprintf(temp, "(print \"%s\\\"", \$.code = gen_code(temp); } { sprintf(temp, "%s", \$5.code); \$.code = gen_code(temp); } { \$.code = \$1.code; } { sprintf(temp, "%s %s", \$.code = gen_code(temp); } { sprintf(temp, "(princ %s)", \$1.code); \$.code = gen_code(temp); } { sprintf(temp, "(princ \$.code = gen_code(temp); } { sprintf(temp, "(print \"%s\\") \$.code = gen_code(temp); } { sprintf(temp, "(print %s) \$.code = gen_code(temp); } </pre>
--	---

3.8. Expresiones y operadores

La jerarquía de operadores sigue la precedencia estándar de C. Se definen varios niveles gramaticales:

- **expr_logica:** operadores lógicos && y || (de menor precedencia).
- **expr_comparacion:** ==, !=, <, >, <=, >=.
- **expr_aditiva:** + y -.
- **expr_multiplicativa:** *, /, %.
- **expr_unaria:** +, -, ! unarios.
- **expr_primaria:** identificadores, números, expresiones entre paréntesis, accesos a vectores y llamadas a funciones.

Como se puede apreciar, estas son las únicas reglas gramaticales que presentan recursividad por la izquierda para mantener la precedencia de operadores y la asociatividad a izquierda (aunque en varios pasos).

De cualquier forma, hemos mantenido las reglas de precedencia de bison en la cabecera que establecen la precedencia de operadores, aunque puede ser un poco redundante, ya que, si bison detecta que esta precedencia se está respetando en la gramática en sí misma, no debería alterarla.

Un ejemplo de las reglas de expresiones es `expr_comparación`, aunque para una mejor visualización de la jerarquía y organización de las reglas, es preferible ver el código en su totalidad.

```

expr_comparacion: expr_aditiva          { $$ = $1; }
                  | expr_aditiva EQ expr_aditiva { sprintf(temp, "(= %s %s)",
$1.code, $3.code);                      $$code = gen_code(temp); }
                  | expr_aditiva NE expr_aditiva { sprintf(temp, "(/= %s %s)",
$1.code, $3.code);                      $$code = gen_code(temp); }
                  | expr_aditiva '<' expr_aditiva { sprintf(temp, "< %s %s)",
$1.code, $3.code);                      $$code = gen_code(temp); }
                  | expr_aditiva '>' expr_aditiva { sprintf(temp, "> %s %s)",
$1.code, $3.code);                      $$code = gen_code(temp); }
                  | expr_aditiva LE expr_aditiva { sprintf(temp, "(<= %s %s)",
$1.code, $3.code);                      $$code = gen_code(temp); }
                  | expr_aditiva GE expr_aditiva { sprintf(temp, "(>= %s %s)",
$1.code, $3.code);                      $$code = gen_code(temp); }
;

```

La tabla de equivalencias de operadores entre C y Lisp se ha detallado en el enunciado y se ha respetado fielmente.

3.9. Vectores (arrays)

El manejo de vectores incluye tres aspectos:

- **Declaración:** `int v[10];` $\rightarrow$ `(setq v (make-array 10)).`
- **Acceso en expresión:** `v[i]` $\rightarrow$ `(aref v i).`
- **Asignación a elemento:** `v[i] = e;` $\rightarrow$ `(setf (aref v i) e).`

Los vectores pueden ser globales o locales. En el caso de vectores locales, también se concatenan con el nombre de la función.

```

expr_primaria: IDENTIF '[' expresion ']'
              { char *var = concat_with_function($1.code);
                sprintf(temp, "(aref %s %s)", var, $3.code); ... }

```

3.10. Directiva `//@ (main)`

El código C de prueba incluye al final una línea `//@ (main)`. El analizador léxico (`yyllex`) reconoce la secuencia `//@` como inicio de código embebido que debe transcribirse literalmente a la salida. Esto permite inyectar la llamada `(main)` en el fichero Lisp generado, de modo que al ejecutarlo con CLISP se ejecuta la función principal. Según el enunciado es importante que el frontend **no** imprima automáticamente `(main)` por sí mismo, ya que eso causaría una doble ejecución, por lo que nos hemos asegurado de que eso sea así. Por este mismo motivo las reglas no se imprimen directamente en el axioma, pero esto lo trataremos en el siguiente apartado.

3.11. Gestión de memoria y código diferido

Aunque la práctica recomendaba evitar el código diferido (acumular grandes cadenas de traducción en atributos), en el frontend se ha utilizado un esquema mixto: las traducciones de las partes pequeñas (expresiones) se van concatenando en atributos `code`, pero las salidas de funciones completas y declaraciones globales se imprimen directamente mediante `printf`. De esta forma se reduce el consumo de memoria dinámica.

De esta forma también se contribuye a la correcta impresión de la directiva mencionada en el anterior punto, ya que, si no lo hubieramos respetado, el axioma imprimiría todo después de la llamada a `main`, causando un error grave.

4. Diseño del Backend (back.y)

El backend toma como entrada el código Lisp producido por el frontend y lo traduce a Forth. A diferencia de las prácticas anteriores, aquí se utiliza traducción directa: cada regla de producción inmediatamente genera la salida Forth correspondiente, sin acumular grandes cadenas. Esto simplifica el diseño y evita problemas de memoria, esto es posible por la notación prefija que utiliza Forth.

La gramática del backend reconoce las siguientes construcciones Lisp:

- Declaraciones de variables: `(setq var expr)`
- Asignaciones: `(setf var expr)`
- Definiciones de funciones: `(defun nombre (params) cuerpo)`
- Llamadas a funciones: `(nombre args...)`
- Estructuras de control: `(if cond then else)`, `(loop while cond do ...)`
- Impresión: `(print ...)`, `(princ ...)`, y secuencias con `(progn ...)`
- Operadores aritméticos, lógicos y relacionales.

4.1. Traducción de variables y asignaciones

En Forth, las variables se definen con la palabra `variable`. Cada vez que aparece una declaración global `(setq var expr)`, se imprime:

```
variable var
código-de-expr var !
```

Es decir, primero se declara la variable y luego se asigna el valor de la expresión. Esto supone que la variable se usará después; si se declarara varias veces, se duplicaría la definición.

Para las asignaciones locales o globales con `(setf var expr)`, se genera directamente: A continuación se muestra un ejemplo representativo de la salida del backend para un programa sencillo.

Entrada C (suma.c):

```
int a, b;
main() {
    a = 5;
    b = a + 2;
    printf("%d", b);
}
```

```
}  
//@ (main)
```

Salida Lisp (generada por trad):

```
(setq a 0)  
(setq b 0)  
(defun main ()  
  (setf a 5)  
  (setf b (+ a 2))  
  (princ b)  
)  
(main)
```

Salida Forth (generada por back):

```
variable a 0 variable a a !  
variable b 0 variable b b !  
: main  
  5 a !  
  a @ 2 + b !  
  b @ .  
;  
main
```

Al ejecutar `gforth` con este código, se imprime el número 7, que es el resultado esperado.

```
código-de-expr var !
```

El código de la expresión se encarga de dejar el valor en la pila, y `var !` lo almacena en la variable.

La regla gramatical correspondiente es:

```
setq_expr: '(' SETQ IDENTIF expr ')'  
  { printf("variable %s ", $3.code);  
    printf("%s %s ! ", $4.code, $3.code); }
```

4.2. Funciones

La definición de una función Lisp (`defun nombre (parámetros) cuerpo`) se traduce a una definición de palabra Forth:

```
: nombre ( params -- ) código-de-cuerpo ;
```

El tratamiento de los parámetros en la implementación actual es más propio de la programación estructurada que de la programación stack based como Forth, pero es la forma en la que hemos podido implementarlo:

1. Primero se declara cada parámetro como variable (con **variable param**) antes de la definición de la palabra. Esto permite usar **param @** y **param !** dentro del cuerpo.
2. A continuación se imprime el encabezamiento : **nombre (params --)** (solo en el caso de funciones que no sean **main**). Para **main** no se imprime la línea de pila, ya que no tiene parámetros.
3. Luego, justo después del encabezamiento, se genera el código que asigna los valores de la pila a las variables locales: para cada parámetro se imprime **param !**. Esto tiene en cuenta que los argumentos deben ser colocados en la pila en el orden inverso al de su declaración, como se espera en Forth.

Un ejemplo: **(defun suma (a b) (+ a b))** se traduce a:

```
variable a
variable b
: suma ( a b -- )
  a ! b !
  a @ b @ +
;
```

La producción gramatical asociada es:

```
func_decl: '(' DEFUN MAIN '(' params ')' { char delims[] = " ";
                                             token = strtok($5.code, delims);
                                             while (token != NULL){
                                                 printf("variable %s\n", token);
                                                 token = strtok(NULL, delims);
                                             }
                                             printf(": %s ", $3.code); }
block ')'
{ printf("; \n"); }

| '(' DEFUN IDENTIF '(' params ')' { char delims[] = " ";
                                     token = strtok($5.code, delims);
                                     while (token != NULL){
                                         printf("variable %s\n", token);
                                         token = strtok(NULL, delims);
                                     }
                                     printf(": %s ", $3.code);
                                     printf("( %s-- )\n", $5.code);

                                     token = strtok($5.code, delims);
                                     while (token != NULL){
                                         printf("%s !\n", token);
                                         token = strtok(NULL, delims);
                                     } }
block ')'
{ printf("; \n"); }
;
```

4.3. Estructuras de control

4.4. Llamadas a funciones

Una llamada a función (**nombre arg1 arg2 ...**) actualmente se traduce simplemente como:

nombre

4.4.1. If

La forma (if cond entonces else) se traduce mediante una estructura anidada de if y else de Forth, pero la implementación real tiene una forma peculiar:

- La regla if_start imprime if después de leer la condición.
- La rama then se espera como (progn ...) (regla if_body), que genera el código del bloque sin imprimir nada adicional.
- Opcionalmente, antes del) de cierre, puede aparecer else seguido de otro (progn ...).

El código generado para (if cond (progn A) (progn B)) es:

```
código-cond if código-A else código-B then
```

Si no hay else, solo se genera if código-A then.

Las producciones gramaticales correspondientes son:

```
statement : '(' SETF IDENTIF expr ')' { printf("%s !\n", $3.code); }
           | '(' PRINT STRING ')' { printf(".\" %s\" cr\n", $3.code); }
           | '(' PRINC STRING ')' { printf(".\" %s\" cr\n", $3.code); }
           | '(' PRINC expr ')' { printf(". \n"); }
           | '(' PROGN printf_sequence ')' { printf("cr\n"); }
           | if_start if_body { printf("else\n"); }
           | if_body ')' { printf("then\n"); }
           | if_start if_body ')' { printf("then\n"); }
           | '(' LOOP WHILE { printf("begin "); }
           | expr { printf(" while \n"); }
           | DO block ')' { printf("repeat\n"); }
           | function_call { ; }
           ;
if_start: '(' IF expr { printf(" if \n"); }
        ;
if_body: '(' PROGN block ')' { ; }
        ;
```

Como se observa, el if se implementa directamente en statement, sin red denominación, lo cual rompe en cierta forma con el estilo que tienen los demás tipos de statement y quizás hubiera sido mejor implementarlo como una regla aparte.

La rama then y else pueden ser bloques (progn ...) o expresiones simples. La producción then_expr y else_expr se encargan de generar el código adecuado.

4.4.2. Loop while

La construcción (loop while cond do ...) se traduce a begin cond while ... repeat. En Forth, begin marca el inicio del bucle, while comprueba la condición (que debe dejar un valor booleano en la pila) y si es verdadero ejecuta el código hasta repeat, que salta de nuevo a begin. Si es falso, sale después de repeat.

Estas reglas se vuelven a implementar sobre `statement`, con las mismas consideraciones que en el apartado anterior.

4.5. Operadores

Lisp Forth	———— ————	(+ a b) a @ b @ +	(- a b) a @ b @ -	(* a b) a @ b @ *	(/ a b) a @ b @ /	(mod a b) a @ b @ mod	(= a b) a @ b @ =	(< a b) a @ b @ <	(and a b) a @ b @ and	(or a b) a @ b @ or	(not a) a @ 0=
--------------	-----------	---------------------	---------------------	---------------------	---------------------	-------------------------	---------------------	---------------------	-------------------------	-----------------------	------------------

4.6. Impresión

14

- `(print "cadena")` → `." cadena" cr`
- `(princ "cadena")` → `." cadena` (sin `cr` explícito, aunque el código imprime `cr` adicionalmente en algunos casos)
- `(princ expr)` → `.` (imprime el número en la cima de la pila, seguido de un espacio y un salto de línea, porque la acción imprime `." \n"`)
- Una secuencia de `princ` dentro de `(progn printf_sequence)` genera los correspondientes `." ..."` o `.`, y al final se añade un `cr`.

El manejo de `printf_sequence` es limitado: solo reconoce `(princ ...)` repetidos, no otros tipos de sentencias.

Regla gramatical:

```

: princ_stmt          { ; }
                        | princ_stmt printf_sequence { ; }
                        ;

princ_stmt : '(' PRINC expr ')'      { printf(". "); }
           | '(' PRINC STRING ')'    { printf(".\" %s\" ", $3.code); }
           ;

```

4.7. Consideraciones sobre la pila

El backend no necesita generar explícitamente operaciones de manipulación de pila (`dup`, `swap`, `drop`) porque la evaluación de expresiones aritméticas en notación postfija ya maneja la pila de forma natural. Por ejemplo, la expresión Lisp `(+ (* a b) c)` se traduce a `a @ b @ * c @ +`, que deja el resultado en la cima sin necesidad de `dup`. Esto es posible porque cada subexpresión se traduce en orden postfijo.

5. Pruebas

Para probar el sistema completo, se ha seguido el siguiente flujo:

1. Compilar el frontend: `bison -d trad.y && gcc -o trad trad.tab.c -lm`
2. Compilar el backend: `bison -d back.y && gcc -o back back.tab.c -lm`
3. Para cada programa de prueba `prueba.c`:
 - `./trad < prueba.c > prueba.lisp`
 - `./back < prueba.lisp > prueba.fs`
 - `gforth prueba.fs`

Se ha verificado que la salida de `gforth` coincide con la salida del programa C original compilado con `gcc` (salvando pequeñas diferencias en los saltos de línea, tal como se indica en el enunciado). También se ha comparado con la salida de `clisp prueba.lisp` para garantizar que el frontend genera Lisp correcto.

Estas pruebas se han llevado a cabo con todas las carpetas para el `trad` y con la carpeta `00` para el `back`, como se indicó.

5.1. Conjunto de pruebas

Se han utilizado los siguientes ficheros de prueba creados por nosotros:

- `01_variables_globales.c` – Declaración y uso de variables globales.
- `02_variables_locales.c` – Variables locales con inicialización.
- `03_printf_puts.c` – Impresión con `printf` y `puts`.

- `04_operadores.c` – Operadores aritméticos, lógicos y de comparación.
- `05_if_else.c` – Estructura condicional.
- `06_while.c` – Bucle `while`.
- `07_for.c` – Bucle `for` con macros `INC`.
- `08_switch.c` – Sentencia `switch`.
- `09_funciones.c` – Definición y llamada a funciones, recursividad.
- `10_vectores.c` – Vectores globales y acceso.
- Además, los ficheros `01_infinite_while_loop.c` hasta `20_function_with_multiple_returns.c` que prueban casos de bloqueo y error (división por cero, acceso fuera de rango, etc.). En muchos de estos últimos, el traductor no detecta los errores ya como hemos comentado en repetidas ocasiones el análisis sintáctico realizado no puede hacer estos análisis, sino que se produce un error en tiempo de ejecución en Lisp o Forth.
- Adicionalmente hemos añadido también los casos límite como declaraciones vacías y asignaciones encadenadas (las cuales no deben funcionar). Cabe destacar que en estas últimas es importante el hecho de que el quinto test prueba que se puede usar `break` fuera de `switch`, considerándolo un pequeño error, ya que no se debería permitir.

5.2. Resultados de las pruebas

Todos los programas de prueba que son semánticamente correctos (sin errores) se traducen correctamente y producen la salida esperada. Algunos casos particulares:

- El bucle infinito `while(1)` se traduce a `(loop while 1 do ...)`, que en Lisp es un bucle infinito (y en Forth también). No hay problema.
- La recursión infinita produce una recursión infinita en Lisp, que eventualmente agota la pila; es un error lógico del programa original.
- La división por cero es un error en tiempo de ejecución de Lisp (y de Forth) y se detecta al ejecutar.
- El acceso fuera de rango en vectores también produce error en Lisp (`AREF: index out of range`).
- La sentencia `break` fuera de `switch` se traduce de igual forma a vacío, por lo que podría considerarse un pequeño error ya que no se debería comportar de esta forma.

En general, el traductor cumple con las especificaciones y genera código ejecutable en ambos entornos.

6. Conclusiones

Se ha completado con éxito la construcción de un traductor de C a Lisp y de Lisp a Forth, cumpliendo todos los hitos solicitados en el enunciado. El frontend maneja variables globales y locales, funciones (con parámetros y retorno), estructuras de control (`if`, `while`, `for`, `switch`), vectores, y entrada/salida básica. El backend convierte las construcciones Lisp más habituales a un subconjunto de Forth que puede ser ejecutado.

En el desarrollo del trabajo se ha seguido un diseño modular y limpio, con una gramática jerárquica y acciones semánticas sencillas. Se han evitado conflictos LALR y se ha prestado especial atención a la correcta gestión de la tabla de símbolos locales a la vez que intentamos minimizar la creación de variables globales y funciones. La integración de ambos componentes ha sido probada con numerosos ejemplos, obteniendo los resultados esperados.

Esta práctica nos ha servido para afianzar de cierta forma más práctica nuestros conocimientos sobre compiladores, Lisp y Forth, al tener que resolver conflictos relacionados con estos.

7. Listado de pruebas y código generado

Los ficheros completos `trad.y`, `back.y` y los programas de prueba se incluyen en los archivos separados `listadoF.pdf`, `listadoB.pdf` y `pruebas.zip`, tal como se solicita en la entrega.